DevOps Laboratory

Assignment 5

Cloud Infrastructure and Server Automation Using AWS and Ansible

Student Name	YUVRAJ SHEMBALE
Roll No / PRN	123B1D045
Date of Submission	

1. Aim

To create a basic cloud automation environment by provisioning an AWS EC2 instance and using Ansible to automate the installation, configuration, and deployment of a web server on it.

2. Introduction

Cloud platforms provide on-demand infrastructure and services that can be provisioned according to application requirements, without the need to purchase and maintain physical hardware. Amazon Web Services (AWS) offers a broad range of such services, including EC2 for virtual servers, S3 for object storage, RDS and Aurora for managed relational databases, ELB for load balancing, and ECS for container orchestration.

While cloud platforms make it easy to provision servers, configuring and maintaining those servers consistently at scale requires automation. Ansible is an agentless automation and configuration-management tool that uses simple, human-readable YAML playbooks to automate tasks such as software installation, configuration, and application deployment across one or many managed servers, connecting to them over SSH.

3. Technologies / Tools Used

Technology	Purpose / Role
AWS EC2	Provides the virtual server (managed node) that Ansible will configure
Ansible	Automation and configuration-management tool used to configure the EC2 instance
SSH	Secure protocol used by Ansible to connect to the managed EC2 instance
YAML	Language used to write the Ansible inventory and playbook files
Nginx	Web server installed and configured on the EC2 instance via Ansible

4. Project / Repository Structure

```
ansible-aws-demo/
■■■ inventory.ini         # defines the EC2 host(s) managed by Ansible
■■■ site.yml              # main Ansible playbook
■■■ ansible.cfg           # Ansible configuration (optional)
■■■ key.pem               # EC2 SSH private key (never commit to version control)
■■■ templates/
    ■■■ index.html.j2     # sample web page template
```

5. Procedure

1. Create or use an existing AWS EC2 instance (e.g., Ubuntu Server) to act as the managed node.

2. Configure secure access to the instance by setting up an SSH key pair and an appropriate security group allowing SSH (22) and HTTP (80) traffic.

3. Install and configure Ansible on the control machine (the local machine or a separate management server).

4. Create an Ansible inventory file listing the EC2 server under a host group, referencing its public IP and SSH key.

5. Verify connectivity to the managed node using an Ansible ad-hoc command such as the ping module.

6. Create an Ansible playbook (site.yml) targeting the web server host group.

7. Use the playbook to install a web server such as Nginx using the appropriate package module.

8. Add tasks to start and enable the web server service so it persists across reboots.

9. Verify that the web server is accessible by browsing to the EC2 instance's public IP address.

10. Modify the playbook to deploy or update a simple web page (e.g., a custom index.html).

11. Execute the playbook again (Ansible playbooks are idempotent) and verify that the updated page is served correctly.

6. Code / Configuration Snippets

6.1 Ansible Inventory (inventory.ini)

```
[webservers]
web1 ansible_host=<EC2_PUBLIC_IP> ansible_user=ubuntu
ansible_ssh_private_key_file=key.pem
```

6.2 Verifying Connectivity (Ad-hoc Command)

```
$ ansible webservers -i inventory.ini -m ping

# Expected output:
# web1 | SUCCESS => {
#     "ansible_facts": { "discovered_interpreter_python": "/usr/bin/python3" },
#     "changed": false,
#     "ping": "pong"
# }
```

6.3 Playbook to Install and Start Nginx (site.yml)

```
- name: Configure Web Server
  hosts: webservers
  become: yes

  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
        update_cache: yes

    - name: Start Nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

6.4 Extended Playbook to Deploy a Custom Web Page

```
    - name: Deploy custom web page
      copy:
        content: |
          <html>
            <head><title>Ansible Demo</title></head>
            <body><h1>Deployed via Ansible on AWS EC2</h1></body>
          </html>
        dest: /var/www/html/index.html
```

6.5 Running the Playbook

```
$ ansible-playbook -i inventory.ini site.yml
```

7. Output / Screenshots

The following screenshots demonstrate successful execution of each major step. Replace each placeholder with the corresponding screenshot captured during the practical.

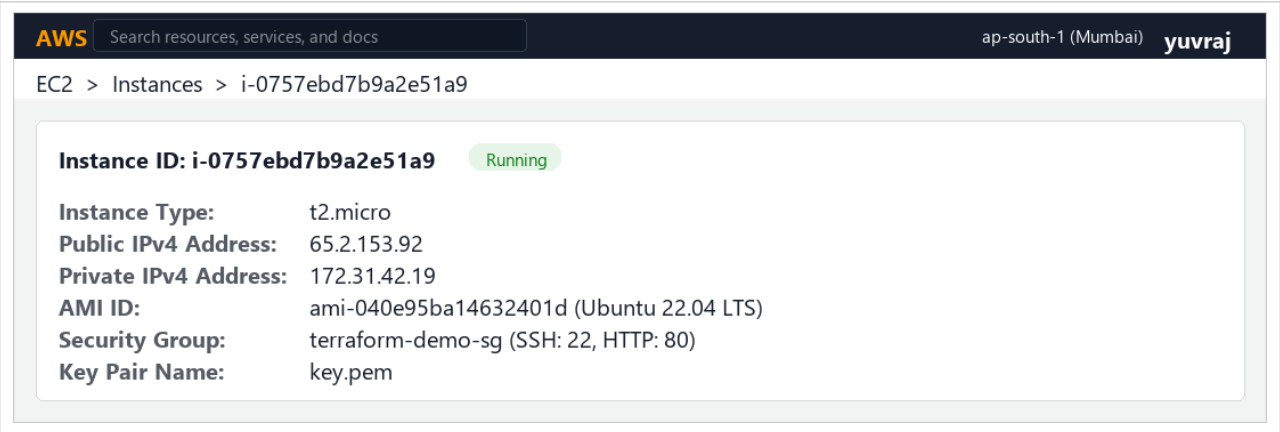


Fig 1: AWS EC2 instance running in the AWS Management Console

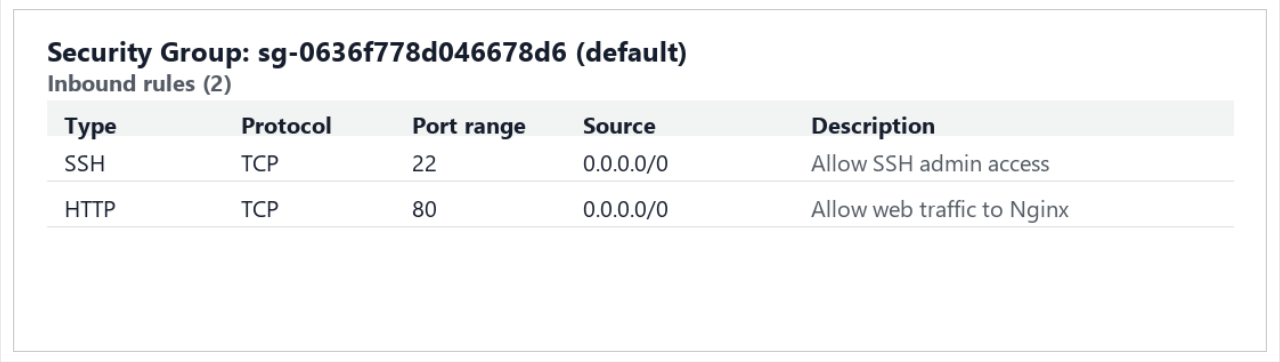


Fig 2: Security group configuration allowing SSH and HTTP access



Fig 3: Successful Ansible ad-hoc ping command output

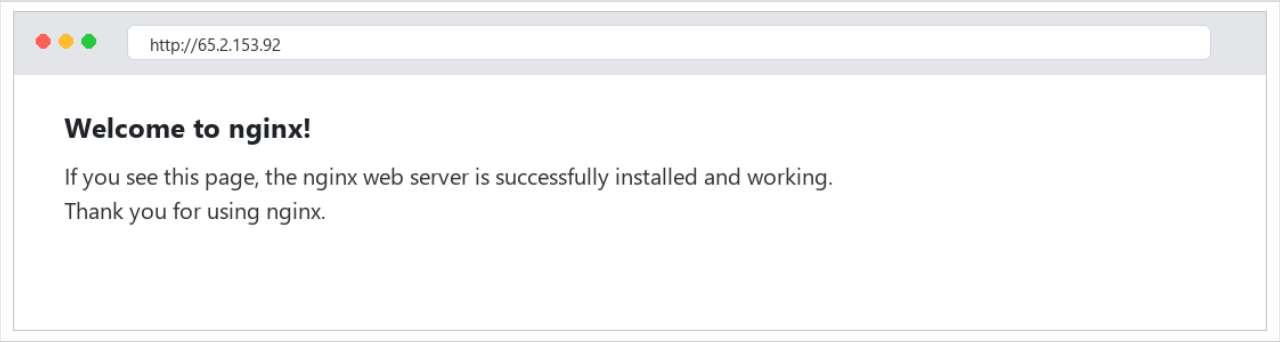


Fig 4: Nginx default/updated page accessed via the EC2 public IP in a browser

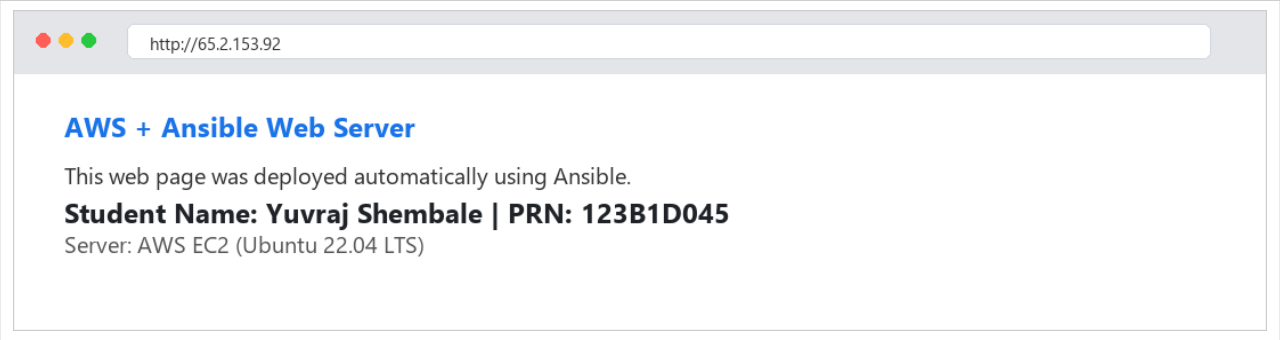


Fig 5: Second playbook run showing idempotent (no unnecessary changes) execution

8. Best Practices

- Use secure authentication (SSH key pairs) when connecting to EC2 instances; avoid password-based SSH access.
- Never expose private keys or credentials in source code or version control.
- Maintain inventories and playbooks under version control for repeatability and auditing.
- Use Ansible modules (e.g., apt, service, copy) instead of unnecessary raw shell commands, for idempotency and clarity.
- Terminate unused EC2 resources after completing the practical to avoid incurring unnecessary cloud costs.

9. Conclusion

AWS provides scalable, on-demand cloud infrastructure, while Ansible enables automated, repeatable configuration and management of the servers running on that infrastructure. In this assignment, an EC2 instance was provisioned and secured, Ansible connectivity was verified, and a complete playbook was used to install, start, and configure an Nginx web server.

Combining cloud infrastructure with configuration automation, as demonstrated here, significantly reduces manual effort and human error, and provides a consistent, repeatable approach to deploying and maintaining applications — a core principle of modern DevOps practice.